Writing a static blog with Typst

Tim Peko (TimerErTim)

Let's dive into the world of Typst and see how we can use it to power this very blog website. We will investigate potential solutions, the custom build process I settled on, and the technologies necessary. For static hosting, highly efficient compression and patching are necessary.
Or in other words: How to properly misuse Typst for everything.

Table of Contents

1 Introduction

Ever since I started programming at around 14 years old I have been fascinated by interactively exploring ideas and finding problems with my mediocre solutions. The projects coming out of that were slowly but almost always for sure forgotten over time, even by myself. In the last few weeks I figured that to be a real shame and that I should start writing down my ideas and solutions to them, especially because technical writing is a skill I enjoy and want to improve.

Because I don't do things by halves, I intend to reignite my online persona **TimerErTim** and use it to cover all these technical/semi-professional aspects of my life. It shall be used as a central point of reference for myself, including videos on YouTube, career and professional information, as well as (probably very irregular) blog posts on my website.

Now, there is one problem: The website does not yet exist. So let's go build it. Let's use Typst for writing the blog posts.

2 Why Typst?

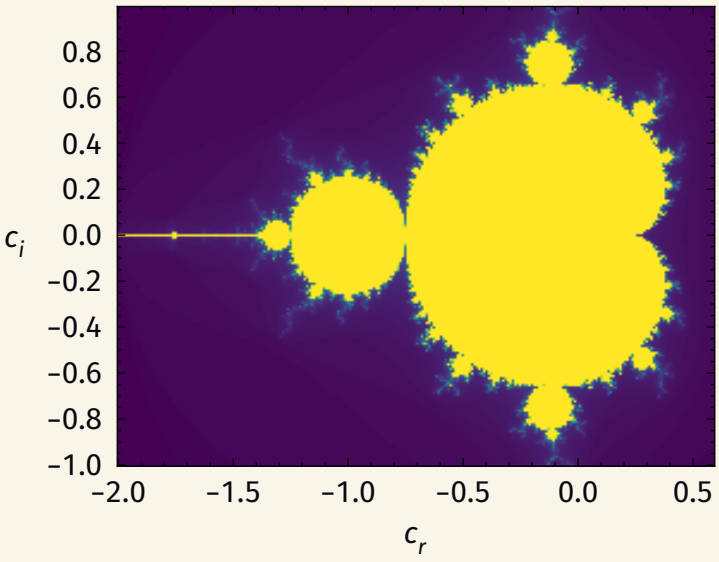
> ***But why not use Markdown or some of the 1000 static site generators out there?***
>
> –You probably

Very good question. There are a few reasons for that, most importantly: **it's boring**.
Writing code is fun, and with our custom implementation only our imagination[1] is the limit.

Furthermore, I really enjoy Typst for its capabilities and its simplicity. It is a joy to write, and the results are beautiful. Show me where Markdown can do this:

$$z_{n+1} = z_n^2 + c \tag{1}$$

Image of Equation 1



[1] Unfortunately artistic creativity is none of my strengths

The above Mandelbrot set is generated entirely in Typst during document compilation with the following code:

```typst
#lq.diagram(
  title: [Image of @mandelbrot-equation],
  xaxis: (
    label: [
      $c_r$
    ],
  ),
  yaxis: (
    label: [
      #show: rotate.with(90deg)
      $c_i$
    ],
  ),
  lq.colormesh(
    linspace(real_range.at(0), real_range.at(1), real_range.at(2)),
    linspace(imag_range.at(0), imag_range.at(1), imag_range.at(2)),
    (x, y) => iterations(x, y),
  ),
)
```

It is very easy to style, and with correct abuse, it can be used to represent the central source of truth for the whole website's look and feel. More on that in Section 5.2.

3 The naive approach

With the tool choice being made, my first idea was to use the typst-ts project of the fantastic Myriad-Dreamin to create a React component that would render the blog post. It is a fantastic tool specifically designed for responsive embedding of Typst documents in web applications.

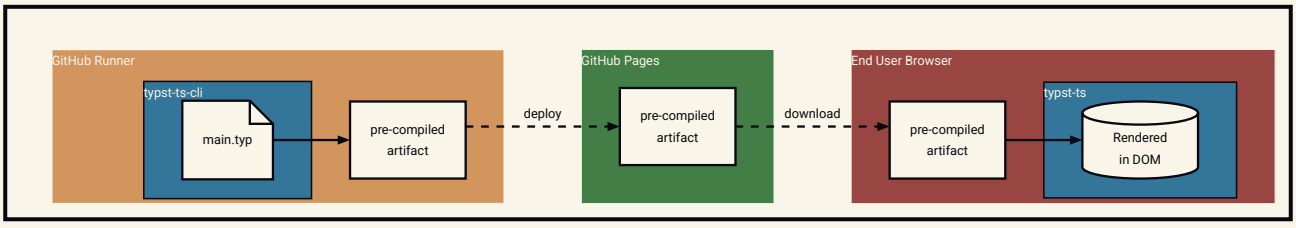
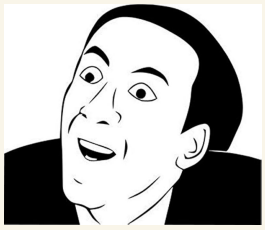


Figure 1: Proposal process for embedding a Typst document in a website

The document is then rendered on the client side by the browser using the `typst-ts` wasm binary… or so I thought.



Turns out we have problems embedding HTML content in Typst and responsiveness is not really responding. HTML content is important because it allows us to do things like embedding an iframe of a totally unrelated Spotify track.

So naturally continuing the search, I dug into shiroa's source code, because as a static site generator using only Typst it was exactly covering my use case. Probably a skill issue, but I could not figure out why the *typst-ts* integration worked beautifully there but not in my own project. It used a similar precompilation process, but the browser rendering was way more low-level and, for me, read like

```
1  ▯▯▯ ▯▯▯ ▯▯▯▯▯▯ ▯▯▯▯▯ ▯ ▯▯▯ ▯▯▯▯
2  ▯▯ ▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯ ▯ ▯▯▯▯▯▯▯▯
3  ▯▯ ▯▯▯▯▯▯ ▯▯▯▯ ▯▯▯▯ ▯▯▯▯▯▯▯▯▯▯ ▯
```

So, without further ado, a custom implementation it is.

4 Doing it ourselves

4.1 Create embeddable SVG documents

Fortunately, the `typst-ts-cli` CLI has a `--format` option that allows us to export the document as an **HTML** page with the whole content rendered as a single **interactive**[2] SVG. A script simply yoinks that out for us.

```
1  <!DOCTYPE html>                                  HTML
2  <html lang="en">
3    <body>
4      <svg
5        xmlns="http://www.w3.org/2000/svg"          } YOINK!
⋮                                                  …
30   </body>
31 </html>
```

Inspecting this extracted SVG reveals how `typst-ts-cli` embeds SVG images in the document

```
#image(bytes("<svg>Hello, world!</svg>"), format: "svg", alt: "!test-alt")
```

[2] Text selection, navigation to headings, etc.

Base64 encoded SVG inside `image` tag — SVG

```
⋮                                                    ⋯
11  <image class="typst-image" width="400" height="300" xlink:href="...IvPjwvc3ZnPg== svg" alt="!test-alt"/>
```

Here, svg is the base64 encoded SVG content. Decoding it reveals our original SVG content: `<svg>Hello, world!</svg>`. Amazing!

A second pass script replaces all occurrences of the `<image>` tag with a specific `alt="!typst-embed-command"` with the base64 decoded SVG content. Now we can precisely control the XML (and by extension HTML) content in our document, ready to embed in the website as normal SVG.

For example embedding the totally unrelated Spotify track mentioned earlier could be accomplished by this code:

Typst

```
1  #xhtml(```html
   <iframe data-testid="embed-iframe" src="https://open.spotify.com/embed/track/7
2  mixpZVqU8AWHvSqOL0wKy?utm_source=generator&amp;si=156c406b2fff4d4a" width="100%"
   height="352" frameBorder="0" allowfullscreen="true" allow="autoplay; clipboard-write;
   encrypted-media; fullscreen; picture-in-picture" loading="lazy"></iframe>
3  ```)
```

where `xhtml` is a helper function handling conversion to SVG's `foreignObject` tag, sizing, etc.

4.2 Embed in website responsively

How to embed fixed size content in a variable width container, like the web browser? Easy fix: Generate multiple width variants of the same document, distribute them and have the browser decide which one to display.

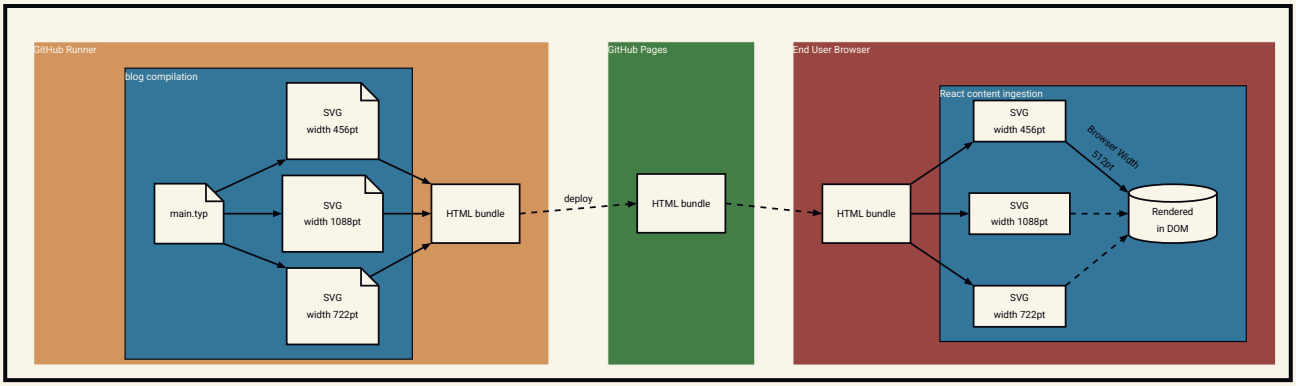


Figure 2: Bundling multiple variants of the blog post with differing widths; the client side browser decides which one to display

We can use the exact same concept for light and dark variants. Now, informing Typst about the target width is as easy as

blog/main.typ Typst

```typst
#let web-page-width = sys.inputs.at("x-page-width", default: none)

#set page(
  width: eval(web-page-width),
  height: auto,
  fill: white.transparentize(100%),
  margin: 0pt,
)
```

and compiling the document with `typst-ts-cli --input "x-page-width=456pt" main.typ` for a width of 456pt.

4.3 Squeeze the bytes out of the payload

6 Size variants × 2 Theme variants × 3 MB (typical size) content SVG size = **36 MB bundle payload**

Hell NO! Every user has to download this payload **for every visit**! We need to heavily compress this ASAP.

Luckily, since the content of all variants stays the same, we can heavily rely on **delta compression** by choosing a single reference variant and for all others calculating the difference to the reference and transmitting that instead.

We will heavily utilize **Brotli** compression because it has great compression ratios, is very fast to decompress and works directly in the browser. It is also easily available with bindings for many programming languages we will require during the build process.

Brotli supports delta compression!

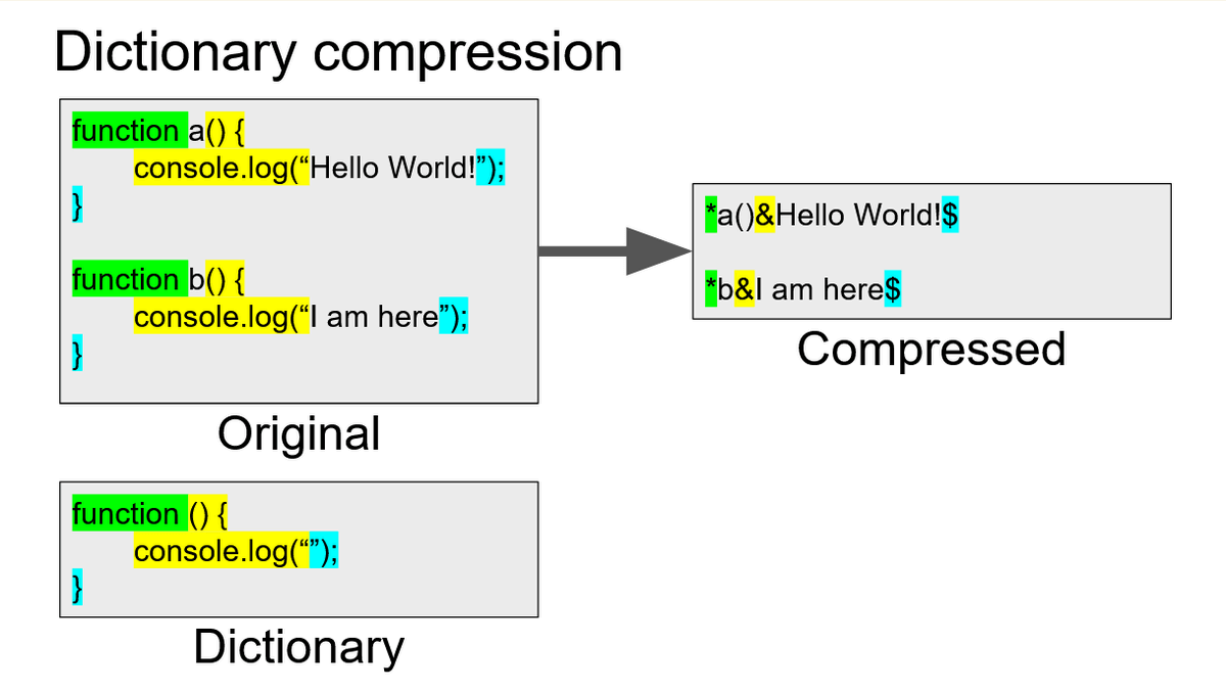


Figure 3: Dictionary compression in **Brotli** [Anna Monus, https://www.debugbear.com/blog/shared-compression-dictionaries]

Modern versions of **Brotli** support dictionary compression, which is a technique that can greatly improve the compression ratio of the compressed data by providing a shared dictionary of common data to compress against.

…or in other words: **A reference variant very similar to the other variants!**

Our reference variant is the dictionary we use for compression and require for decompression.

The reference variant is compressed without any delta compression: 2.4 MB → **866 KB**. Not that bad! A simple build script using NodeJS's **zlib** should handle all the delta compression for us. And indeed, compression ratios are insane now, with delta compressed variants only taking up **2 KB** each.

4.4 Not so fast young man!

Let's go ahead and implement decompression and DOM node ingestion on the browser side. Simple enough, add a corresponding package, implement variant selection and…

```
  791 |               const wordLength = this.copyLength;
  792 |               if (wordLength > 31) {
> 793 |                 throw new BrotliDecoderAllocRingBuffer1Error(
      |                       ^
  794 |                   "Invalid backward reference",
  795 |                 );
  796 |               } (app/error.tsx:15:13)
```

F**K!**

Browser-side Brotli dictionary support is limited

I am sure this is partially a user error, but it turns out **zlib**'s dictionary support is more flexible than the browser-side **Brotli** decoder.

Using the *cleartext* reference SVG variant as dictionary

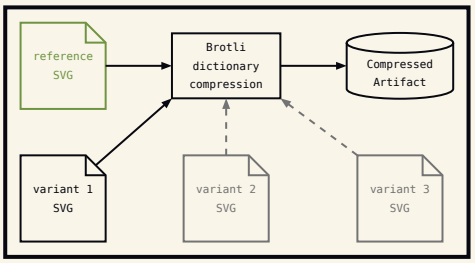
- in **zlib** for compression ✓ works
- in the browser-side **Brotli** decoder ✗ does not

We'd need to preprocess the dict for browser usage somehow.

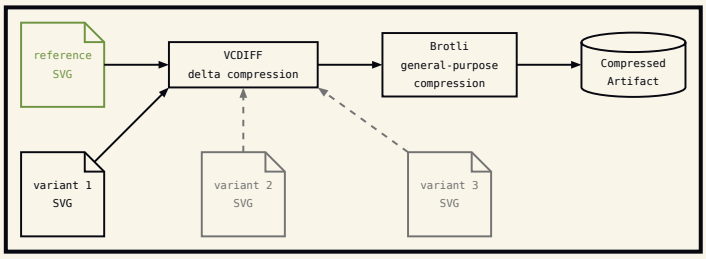
After not figuring it out for one day, I abandoned this concrete approach. I already have a backup idea in mind: Use the same delta compression algorithm as **Git**… **VCDIFF**!

4.5 VCDIFF to the rescue!

Instead of relying on **Brotli**'s dictionary support, we can use **VCDIFF** to create delta patches between the reference variant and the other variants, and then use **Brotli** to compress the delta patches.



Listing 1: Naive **Brotli** dictionary approach



Listing 2: Compression approach with delta patches from **VCDIFF**

VCDIFF is also great, because there are reliable pure browser-side implementations for decoding it.

We use Python with uv in the build scripts for easily reproducible builds.

Both **open-vcdiff** (Google) and **xdelta3** have Python bindings, but they don't work with the latest Python versions, only at most *3.8*. But getting the latest **uv** version to support Python 3.8 was impossible for me. Sigh…

Fortunately, there are up-to-date **xdelta3** bindings for **Rust**, so we quickly write a **PyO3** wrapper around it and use **Maturin** for installing in the script:

libs/xdelta3/src/lib.rs

```rust
//! PyO3 bindings for [xdelta3](https://docs.rs/xdelta3/latest/xdelta3/) (VCDIFF).
use ::xdelta3 as xdelta3_rs;

#[pyfunction]
#[pyo3(signature = (reference, target))]
fn encode(reference: &[u8], target: &[u8]) -> PyResult<Vec<u8>> {
    xdelta3_rs::encode(target, reference)
        .ok_or_else(|| PyValueError::new_err("xdelta3 encode failed"))
}

#[pyfunction]
#[pyo3(signature = (reference, patch))]
fn decode(reference: &[u8], patch: &[u8]) -> PyResult<Vec<u8>> {
    xdelta3_rs::decode(patch, reference)
        .ok_or_else(|| PyValueError::new_err("xdelta3 decode failed"))
}

#[pymodule]
fn xdelta3(m: &Bound<'_, PyModule>) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(encode, m)?)?;
    m.add_function(wrap_pyfunction!(decode, m)?)?;
    Ok(())
}
```

That's all there is to it. We can set this required dependency in **uv** scripts using the frontmatter:

```python
#!/usr/bin/env -S uv run --script
# /// script
# requires-python = "≥3.8"
# dependencies = [
#     "xdelta3",
# ]
#
# [tool.uv.sources]
# xdelta3 = { path = "../../../libs/xdelta3" }  # Path from this file's directory
# ///
import xdelta3
⋮                                                …

with open(filename, "rb") as f:
  file_bytes = f.read()

delta_bytes = xdelta3.encode(reference_bytes, file_bytes)
⋮                                                …
```

And now, finally, after almost no hassle at all, the build process is complete! The client side decompression and rendering works as evidenced by you reading this.

Stats

static files @ out/blog/2026-blog-with-typst/

```
1.5M 2026-blog-with-typst.pdf
3.0M __next._full.txt
5.5k __next._head.txt
7.3k __next._index.txt
 878 __next._tree.txt
3.0M __next.blog.$d$blog-name.__PAGE__.txt
 599 __next.blog.$d$blog-name.txt
 599 __next.blog.txt
3.0M index.html
3.0M index.txt
```

We managed to squeeze all variants, content, scripts, … into a single **3 MB** HTML payload!

What this 3 MB includes

While still quite a lot for a single webpage, it is one hell of an improvement over the original 36 MB we would need uncompressed. And remember, this single HTML payload delivers everything for

- every theme variant
- every size variant
- every script needed for interactivity
- every 3rd party content such as images
- injecting the content into DOM
- the normal webpage content besides the blog entry

5 How does … work?

If that question is floating around in your head, you can look around this chapter for answers.

5.1 PDF download

When you click the “download” button on the website, the browser fetches a precompiled PDF version of this blog post.

Blogpost Styling: Either web optimized or PDF optimized

Typst

```
#let targets-web = sys.inputs.at("x-target", default: "classic") == "web"

#let web-or(
  web-variant,
  classic-variant,
) = if targets-web {
  web-variant
} else {
  classic-variant
}

#show: web-or(
  web-template,
  pdf-template,
)
```

The build process injects the correct `#sys.inputs` for the correct artifact type, same as in Section 4.2.

5.2 Centralized look and feel

Because everything is organized as a monorepo, we can define a central file for the colors of our look and feel:

colors.typ

```typst
#let colors = (
  "light": (
    "base": oklch(98%, 1%, 265deg),
    "foreground": oklch(44%, 4%, 279deg).darken(50%),
    "accent": rgb("#62a123").oklch(),
    "neutral": oklch(86%, 1%, 268deg),
    "surface": oklch(93%, 1%, 265deg),
    "overlay": oklch(91%, 1%, 265deg),
    "border": oklch(71%, 2%, 275deg),
    "danger": oklch(55%, 22%, 20deg),
    "warning": oklch(71%, 15%, 68deg),
    "success": oklch(63%, 18%, 140deg),
    "info": oklch(68%, 14%, 235deg), // Info is also used for links
  ),
  "dark": (
    "base": oklch(18%, 2%, 284deg),
    "foreground": oklch(88%, 4%, 272deg).lighten(50%),
    "accent": rgb("#426E17").oklch(),
    "neutral": oklch(40%, 3%, 280deg),
    "surface": oklch(22%, 3%, 284deg),
    "overlay": oklch(24%, 3%, 284deg),
    "border": oklch(55%, 3%, 277deg),
    "danger": oklch(76%, 13%, 3deg),
    "warning": oklch(92%, 7%, 87deg),
    "success": oklch(86%, 11%, 143deg),
    "info": oklch(85%, 8%, 210deg), // Info is also used for links
  ),
)
```

These are combined with layout and font definitions into `themes`, which in turn is imported by this very blog entry.

Exporting for third party consumption

export.typ

```typst
#metadata(
  (themes,).map(_convert-to-js-units).map(_convert-to-hex-colors).first(),
) <themes>
```

This metadata field converts all units in our data structure to standard units and exposes them to third-party tools, such as our build tool:

```
typst query export.typ "<themes>" --field value --one --pretty > themes.json
```

The website build pipeline takes these theme values and generates a CSS file, defining the whole look and feel of the website:

website/tasks/scripts/generate-theme.mjs JavaScript

```
const css = `/* Generated from look-and-feel/themes.json — do not edit */
:root,
.light {
${themeVars(themes.light)}
}

.dark {
${themeVars(themes.dark)}
}

@theme inline {
  --color-background: var(--theme-base);
  --color-foreground: var(--theme-foreground);
  --color-accent: var(--theme-accent);
  --color-muted: var(--theme-muted);
  --color-surface: var(--theme-surface);
⋮                                                    …
  --leading-large: ${layout.lineHeight.large}px;

  --radius-sm: ${layout.radius.small}px;
  --radius-md: ${layout.radius.medium}px;
  --radius-lg: ${layout.radius.large}px;
}
`;
writeFileSync("src/app/theme.generated.css", css);
```

You can use imports with PostCSS!

website/src/app/globals.css CSS

```
@import "tailwindcss";
@import "./theme.generated.css";
⋮                                                    …
```

Tailwind registers the generated CSS file and uses the theme variables in its config.

6 Potential improvements

The solution is not yet perfect. While pretty much fully functional, there are things left to improve.

6.1 Remove iframe reloading upon resize

Because we actually insert a new DOM child and remove the previous one upon resize, the iframe is considered to be different by the browser, which causes a reset of its state, effectively canceling any ongoing playback.

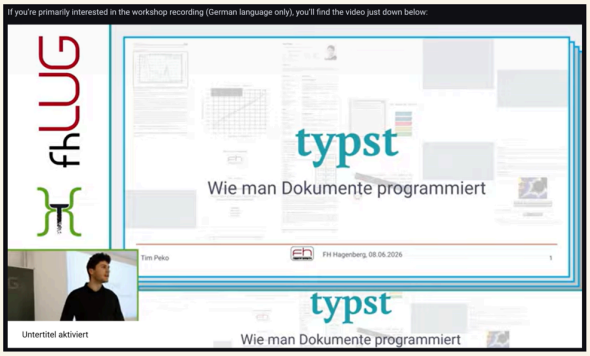


Figure 4: Playing YouTube video before resize

Figure 5: Thumbnail after resize

6.2 More Website integration

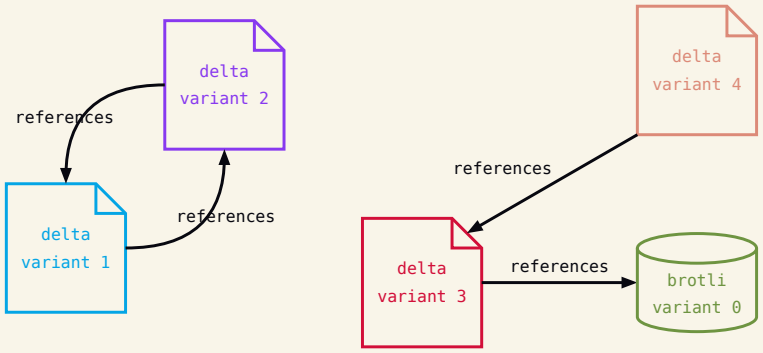
The current solution only writes the following metadata to the filesystem during the export process:

```json
{
  "author": [
    "Tim Peko (TimerErTim)"
  ],
  "title": "Writing a static blog with Typst",
  "description": "Let's dive ... Typst for everything.",
  "keywords": [
    "Typst",
⋮                                                    …
    "Brotli",
    "Vcdiff"
  ]
}
```

NextJS is therefore unable to include any complex content in the description of the blog post, such as images or videos.

6.3 Better compression

Even though the current solution is already pretty efficient, there is still room for improvement. We could, for example, search for the best reference variant for each possible target variant and then store what reference was used for every individual target variant. This is not trivial, because we need to prevent cyclic references. Imagine the following situation:



Listing 3: Cyclic delta compression references: Files represent delta patches, cylinders represent only brotli compressed full variants.

Here a primitive algorithm would see variant 1 has the best compression ratio delta compressing against variant 2 as reference, and variant 2 with variant 1 as reference. It would compress them both against each other's cleartext, uncompressed format. But now, we can never decompress them again because we have no way to restore the original uncompressed reference for one of these two. variant 3 and variant 4 would work fine since we can trace back the reference hierarchy to variant 0.

Let's exploit XML's strict structure

Now we are in Min-Maxing territory. We could also utilize an XML optimized compression algorithm like **EXI** to compress the SVG instead of general-purpose compression algorithms like **Brotli**. SVGO is a tool that can optimize SVGs for size and performance and could be used in the compression pipeline.

Our SVGs are currently incompatible with SVGO, so further investigation is necessary.